

OAuth 2.0

Key Concepts

- **OAuth 2.0** — authorization framework (not authentication); grants third-party apps limited access to resources without sharing passwords
- **Authorization Code** — short-lived, single-use proof of consent; travels through browser; cannot call any API
- **Access Token** — actual key to call APIs; short-lived (15min–1hr); fetched backend-to-backend
- **Refresh Token** — long-lived (days/weeks); used to get new access tokens without re-login
- **Scope** — defines what the client is allowed to access (`email profile calendar.read`)
- **State** — random string to prevent CSRF during redirect flow

The 4 Actors

Actor	Who	Example
Resource Owner	The user	You
Client	App requesting access	Spotify
Authorization Server	Issues tokens	Google Auth
Resource Server	Holds protected data	Google APIs

Authorization Code Flow

1. Client redirects user to Authorization Server
GET /authorize?
response_type=code&client_id=X&redirect_uri=Y&scope=email&state=Z
2. User logs in and approves on Authorization Server
3. Auth Server redirects back with Authorization Code
https://spotify.com/callback?code=AUTH_CODE&state=Z
4. Client backend exchanges code for tokens (server-to-server)
POST /token
code=AUTH_CODE
client_id + client_secret
grant_type=authorization_code
5. Auth Server returns:
{ access_token, refresh_token, expires_in, scope }
6. Client calls Resource Server with Access Token
GET /userinfo
Authorization: Bearer access_token

Why Two Steps? (Authorization Code → Access Token)

- Authorization Code travels through the **browser** (URL redirect) — unsafe (history, logs, referrer headers)
- Access Token is fetched **backend-to-backend** using `client_secret` — never touches the browser
- This keeps the actual API key out of the browser entirely

```
BROWSER (unsafe):  Authorization Code only
BACKEND (safe):    Access Token + Refresh Token
```

Token Lifetimes

Token	Lifetime	Purpose
Authorization Code	~10 minutes, single use	Proof of consent, exchanged for tokens
Access Token	15 min – 1 hour	Calls APIs
Refresh Token	7–90 days	Gets new access token when expired

Refresh Flow

```
POST /token
grant_type=refresh_token
refresh_token=...
client_id + client_secret

Response: { new access_token, new refresh_token }
```

Industry Examples

- **"Sign in with Google/GitHub/Facebook"** — OAuth authorization code flow
- **Slack** — third-party apps post messages using OAuth scopes
- **Dropbox** — apps read/write files via OAuth-scoped access tokens
- **Stripe** — Connect platform uses OAuth to act on behalf of merchants

Interview Questions

Q: What problem does OAuth 2.0 solve?

Lets third-party apps access user resources without ever seeing the user's password. User grants limited, scoped access — the app gets a token, not credentials.

Q: Why is there an authorization code AND an access token? Why not just return the access token directly?

The authorization code travels through the browser URL — which is insecure. The access token is fetched backend-to-backend using the client secret, so it never touches the browser. This two-step process keeps the actual API key out of the browser.

Q: What is a scope?

A scope defines the specific permissions being requested — e.g., `email`, `profile`, `calendar.read`. The user sees and approves these during the consent step. Tokens are only valid for the approved scopes.

Q: OAuth 2.0 is authorization — how do you get identity from it?

You need OpenID Connect (OIDC), which sits on top of OAuth 2.0 and adds an ID Token containing user identity claims.